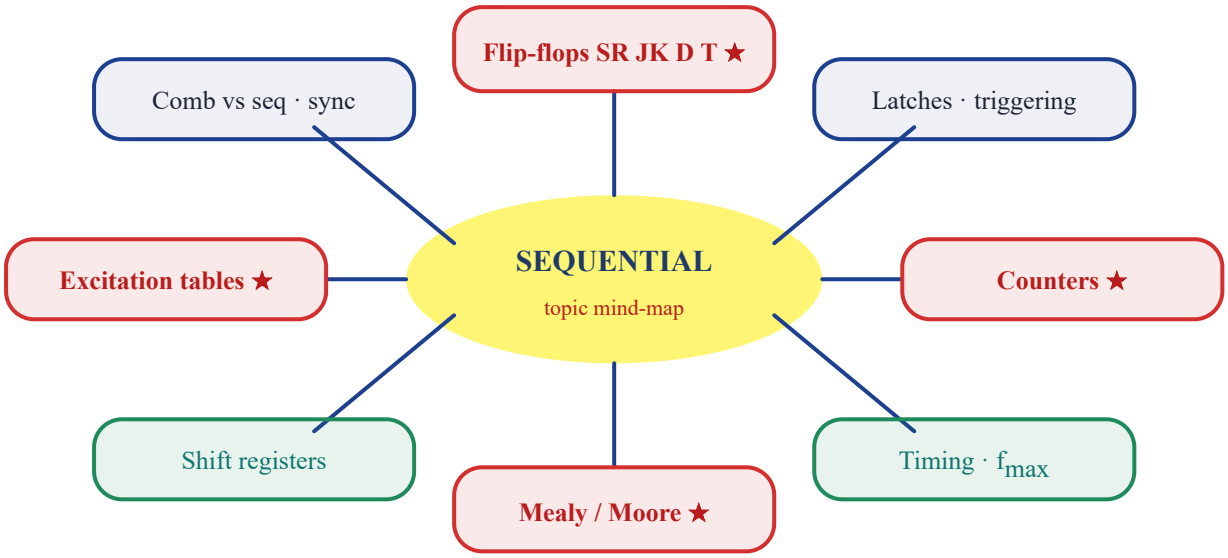
Sequential Circuits

Latches · Flip-flops · Registers · Counters · Mealy/Moore: exam-ready in 12 sheets



↑ the whole topic on one page: red = high-yield, green = method / applied

Syllabus coverage

"**Flip-flops, race around problems**; Counters: **asynchronous counters, synchronous counters** and their applications; Synchronous and asynchronous logic design; **State diagram, Mealy and Moore machines**; State minimizations and assignments; Pulse mode logic; Fundamental mode design". Gates are in the Logic Gates notes, K-maps in the Boolean Algebra notes; MUX/decoders/adders belong to Combinational Circuits.

High-yield list

① $Q^+ = JQ' + K'Q$
② SR forbidden state
③ excitation tables
④ race-around → M-S
⑤ ring n · Johnson 2n
⑥ mod-N: $\lceil \log_2 N \rceil$ FFs
⑦ Mealy vs Moore

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

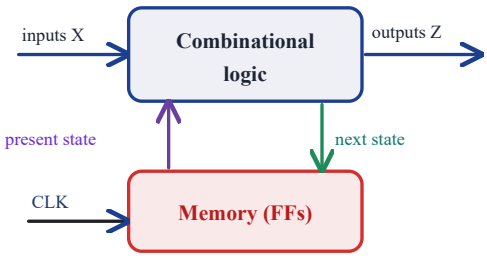
Contents

✓ Combinational vs sequential, clock … p. 2
✓ SR latch, gated latches …… p. 3
✓ Triggering, characteristic tables … p. 4
✓ Excitation, race-around, M-S … p. 5
✓ Conversions, timing, f_{max} …… p. 6
✓ Registers & shift registers …… p. 7
✓ Counters I: ripple & synchronous … p. 8
✓ Counters II: ring, Johnson, design … p. 9
✓ State machines, Mealy/Moore … p. 10
✓ Night-before sheet …… p. 11
✓ Practice MCQs + written …… p. 12

1 Combinational vs Sequential

Definition: A **sequential circuit** is combinational logic **plus memory**: its outputs depend on the present inputs **and** on the present state, i.e. on the **past history** of inputs. The memory elements are latches or flip-flops, connected through a **feedback** path.

	Combinational	Sequential
Output from	inputs only	inputs + state
Memory, feedback	none	**yes** (FFs)
Clock	not needed	usually
Described by	truth table	state table
Examples	adder, MUX	register, counter



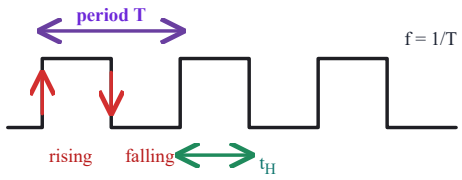
the feedback loop = memory

Memory trick: "Sequential = **remembers**". Ask: can the same input give two different outputs? If yes, the circuit has state, so it is sequential.

2 Synchronous vs Asynchronous

	Synchronous	Asynchronous
State changes	only at the **clock edge**	as soon as an input changes
Memory element	clocked flip-flops	unclocked latches / gate-delay loops
Design & testing	**easy, predictable**	hard: races, hazards
Speed	limited by the clock period	can be faster, lower power
Examples	sync counter, shift register, CPU	SR latch, ripple counter, handshake

- **Fundamental mode** (async): level inputs, only **one** changes at a time, and only when the circuit is stable
- **Pulse mode** (async): inputs are pulses, one at a time, no clock
- **Critical race**: two state bits change together and the result depends on which wins; cure: a one-bit-change state assignment

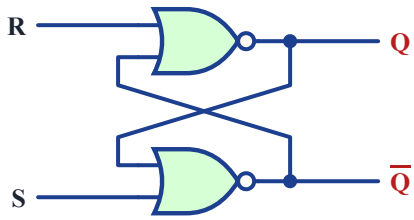


clock: period, edges, duty cycle

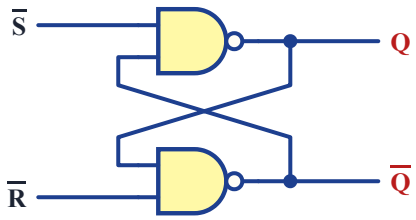
Clock: f = **1/T** · duty cycle = $t_H/T \times 100\% \cdot T = 20$ ns → f = 50 MHz. A flip-flop is a **bistable multivibrator** (2 stable states → **1 bit**); monostable = one-shot pulse; astable = no stable state, a clock oscillator (555 timer).

3 The SR Latch: the Basic Memory Cell

Definition: A **latch** is two cross-coupled NOR or NAND gates, each output feeding the other's input: two stable states, Q = 1 (set) and Q = 0 (reset), with $\bar{Q} = Q'$.



NOR latch: active-HIGH



NAND latch: active-LOW

S	R	Q^+	Action
0	0	Q	hold (no change)
0	1	0	reset
1	0	1	set
1	**1**	?	**forbidden**: $Q = \bar{Q} = 0$

$\bar{S}$	$\bar{R}$	Q^+	Action
1	1	Q	hold (no change)
1	0	0	reset
0	1	1	set
0	**0**	?	**forbidden**: $Q = \bar{Q} = 1$

NOR: $Q = (R + \bar{Q})'$, $\bar{Q} = (S + Q)'$ · NAND: $Q = (\bar{S} \cdot \bar{Q})'$, $\bar{Q} = (\bar{R} \cdot Q)'$ · each uses **2 gates**

Exam trap: Why is S = R = 1 (NOR) "forbidden"? ① Q and $\bar{Q}$ become **equal**, so $\bar{Q}$ is no longer the complement; ② if both inputs then return to 0 **together**, the latch races and the final state is **unpredictable**. For the NAND latch the bad input is **00**, and **11** is hold.

Memory trick: "**NOR hates two 1s, NAND hates two 0s.**" NOR inputs are active-HIGH (idle at 0); NAND inputs are active-LOW (idle at 1, drawn with bars).

4 Gated (Clocked) Latches

Gated SR latch: two steering NANDs in front of a NAND latch (**4 NAND** gates). EN = 0 → hold; EN = 1 → behaves as an SR latch (S = R = 1 still forbidden).

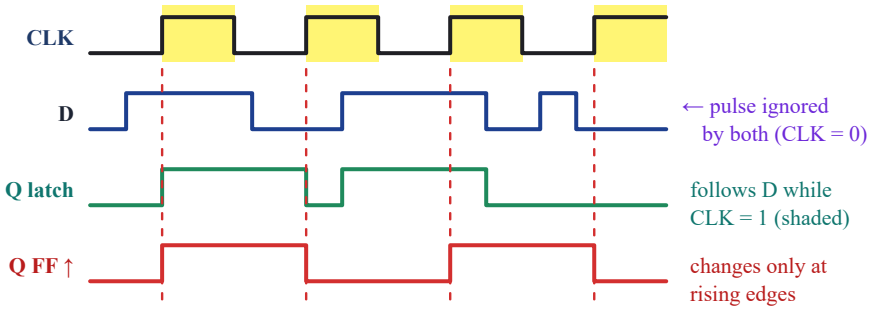
Gated D latch: S = D, R = D′ (add 1 inverter), so **S = R = 1 can never occur**. EN = 1 → **Q = D** (**transparent**); EN = 0 → Q is latched.

EN	D	Q^+	Mode
0	X	Q	latched (hold)
1	0	0	transparent
1	1	1	transparent

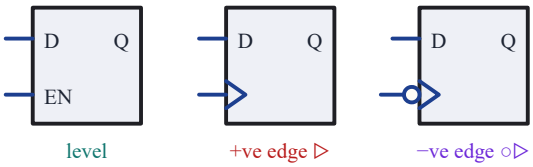
IT link: SR latch = **switch debouncer**. The 74LS373 octal transparent latch, enabled by ALE, separates the 8085's multiplexed AD0–AD7 bus.

5 Level vs Edge Triggering

Latch vs flip-flop: A **latch** is **level-triggered**: it follows its input for the whole time EN is active. A **flip-flop** is **edge-triggered**: it samples its input only at the clock edge, rising (**positive**, 0 → 1) or falling (**negative**, 1 → 0).



same D and CLK: the latch passes the 2nd-pulse change, the flip-flop does not



Symbol trick: ▷ on the clock pin = edge-triggered; a **bubble** + ▷ = falling edge; no ▷ = level (latch). A ┐ mark on the outputs = master-slave (pulse-triggered).

6 Characteristic Tables & Equations

SR flip-flop		
S	**R**	**Q^+**
0	0	Q (hold)
0	1	0 (reset)
1	0	1 (set)
1	1	invalid
$Q^+ = S + R'Q$, SR = 0		

JK flip-flop		
J	**K**	**Q^+**
0	0	Q (hold)
0	1	0 (reset)
1	0	1 (set)
1	1	**Q′ (toggle)**
$Q^+ = JQ' + K'Q$		

D flip-flop	
D	**Q^+**
0	0
1	1
$Q^+ = D$ (data / delay)	

T flip-flop	
T	**Q^+**
0	Q (hold)
1	Q′ (toggle)
$Q^+ = T \oplus Q$ = TQ′ + T′Q	

Derivation ✍️: SR: $Q^+ = 1$ for (S, R, Q) = 001, 100, 101; 110 and 111 are don't-cares → K-map → **$Q^+ = S + R'Q$**. JK: $Q^+ = 1$ for (J, K, Q) = 001, 100, 101, 110 → **$Q^+ = JQ' + K'Q$**.

7 Excitation Tables (for Design)

Definition: The **characteristic table** answers "given the inputs, what is Q^+?" (analysis). The **excitation table** answers the reverse: "to move from Q to Q^+, which inputs are needed?" (design of counters and state machines).

$Q \to Q^+$	S	R	J	K	D	T
$0 \to 0$	0	X	0	X	0	0
$0 \to 1$	1	0	1	X	1	1
$1 \to 0$	0	1	X	1	0	1
$1 \to 1$	X	0	X	0	1	0

JK has the **most X's** (don't-cares), so JK designs need the simplest input logic; D needs no excitation table at all ($D = Q^+$) but usually more gates.

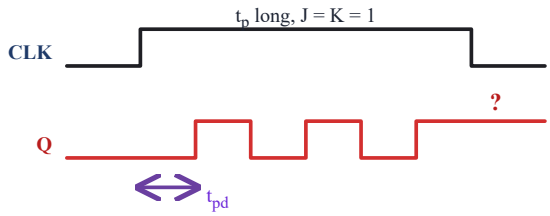
Memory trick: **D column = Q^+ column**. **T column = $Q \oplus Q^+$** (1 whenever it changes). **JK:** when Q = 0 only J matters ($J = Q^+$, K = X); when Q = 1 only K matters ($K = Q^{+\prime}$, J = X).

Exam trap: Don't mix them: "J = K = 1 → toggle" is **characteristic**; "0 → 1 needs J = 1, K = X" is **excitation** ($K = X$, not $K = 0$).

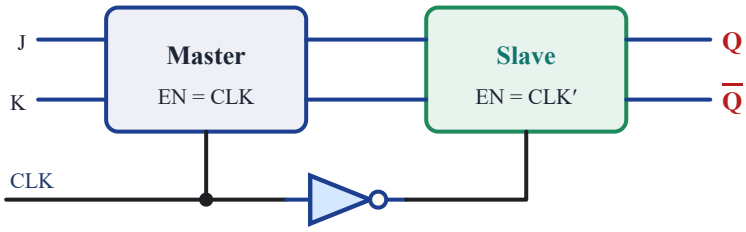
8 Race-Around & the Master-Slave Flip-Flop

Race-around condition: in a **level-triggered** JK with **J = K = 1**, if the clock pulse width t_p is longer than the propagation delay t_{pd}, the output toggles again and again during one pulse, so the final state is **unknown**.

Avoided when $\mathbf{t_p < t_{pd} < T}$, which is hard to guarantee. Real cures: **master-slave** or **edge-triggered** flip-flops.



Q toggles every t_{pd} while CLK = 1



Q, $\bar{Q}$ also feed back to the master (not drawn)

1. CLK = 1: the **master** follows J, K; the slave is disabled, so Q is frozen
2. CLK → 0: the master freezes, the **slave copies** it → Q changes once, after the falling edge; Q cannot change during the pulse → **no race-around**

Drawback: 1s-catching: a glitch on J/K while CLK = 1 is caught by the master; edge-triggered FFs (7474 D, 74LS76 JK) avoid it.

9 Flip-Flop Conversions

Method: ① characteristic table of the FF you **want** (its inputs + Q → Q^+) ② add the **excitation** columns of the FF you **have** ③ K-map each available input in terms of the wanted inputs and Q ④ draw the logic in front of the available FF.

Have → want	Logic	Have → want	Logic
JK → T	**J = K = T**	D → T	**D = T ⊕ Q**
JK → D	J = D, K = D′	D → JK	D = JQ′ + K′Q
SR → D	S = D, R = D′	D → SR	D = S + R′Q
SR → JK	S = JQ′, R = KQ	T → D	T = D ⊕ Q
SR → T	S = TQ′, R = TQ	T → JK	T = JQ′ + KQ

Worked example: SR → JK ✍️:

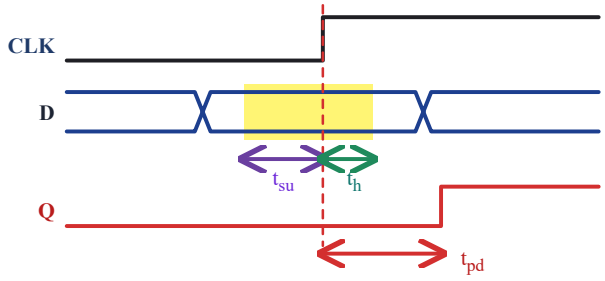
Q	J	K	Q^+	S	R
0	0	X	0	0	X
0	1	X	1	1	0
1	X	0	1	X	0
1	X	1	0	0	1

(8 rows compressed.) S = 1 only for Q = 0, J = 1 → **S = JQ′**; R = 1 only for Q = 1, K = 1 → **R = KQ**. S·R = 0 always, so no forbidden input.

Memory trick: "D wants Q^+, T wants the change": to make **anything** from D, feed it the wanted characteristic equation; to make anything from T, feed it Q ⊕ Q^+.

10 Timing Parameters & Maximum Clock

- **Setup t_{su}**: D stable **before** the edge
- **Hold t_h**: D stable **after** the edge
- **t_{pd}**: clock-to-Q delay (t_{PLH}, t_{PHL})
- **t_w**: min. CLK pulse width



D must be still inside the shaded window

Max clock: $T_{min} = t_{pd(FF)} + t_{comb(max)} + t_{su}$ (+ skew) → $\mathbf{f_{max} = 1 / T_{min}}$

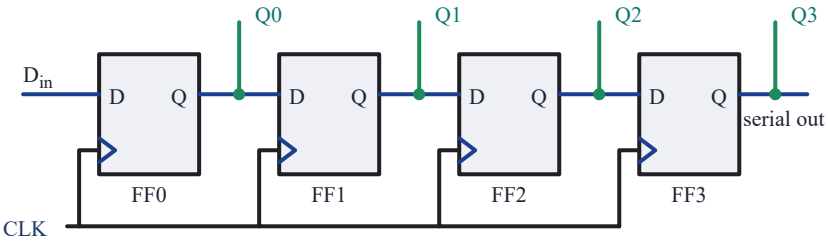
Hold check: $t_{pd(FF)} + t_{comb(min)} \geq t_h$ · **Example** ✍️ 10 + 15 + 5 = 30 ns → $f_{max} \approx$ **33.3 MHz**

Async inputs PRE′ (Q = 1), CLR′ (Q = 0): **active-LOW**, **override the clock**; both 0 not allowed (7474: dual +ve-edge D with PRE′/CLR′).

Exam trap: t_{su} / t_h violated → **metastability** (Q stuck between 0 and 1). Fix: a **2-FF synchroniser**.

11 Registers & Shift Registers

Definition: A **register** is a group of **n flip-flops** (usually D) sharing one clock; it stores an **n-bit** word. A **shift register** also moves every bit one place per clock pulse (each FF's Q drives the next FF's D).



4-bit shift-right register: SISO (serial out) or SIPO (green taps)

Type	In → out	Load n bits	Read out	Typical use · IC
SISO	serial → serial	**n** clocks	n − 1 more clocks	time delay · 7491 (8-bit)
SIPO	serial → parallel	n clocks	at once	UART receiver · 74164
PISO	parallel → serial	1 clock	n − 1 shifts	UART transmitter · 74165
PIPO	parallel → parallel	1 clock	at once	buffer, storage register

Worked example ✍️: Shift 1, 0, 1, 1 (in that order) into a cleared 4-bit SIPO:

CLK	Din	Q0 Q1 Q2 Q3
0	–	0 0 0 0
1	1	1 0 0 0
2	0	0 1 0 0
3	1	1 0 1 0
4	1	**1 1 0 1**

The first bit in is now at Q3 (serial out).

74194: 4-bit universal (bidirectional)		
S1	**S0**	**Mode**
0	0	hold (no change)
0	1	shift right ($Q_A \rightarrow Q_D$)
1	0	shift left ($Q_D \rightarrow Q_A$)
1	1	**parallel load**

CLR′ is asynchronous, active-LOW. **Universal** = shift left + shift right + parallel load (+ hold).

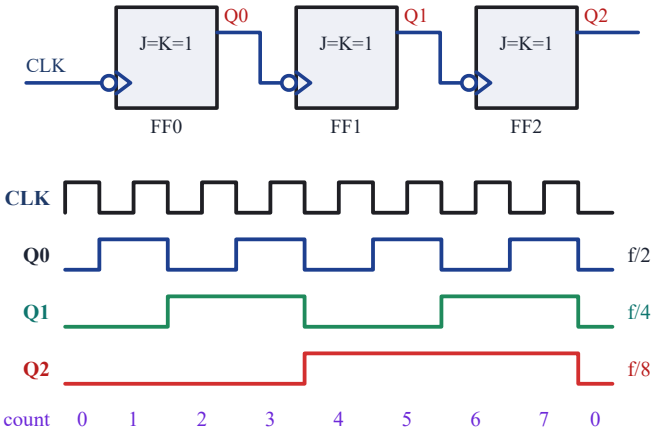
Exam trap: An n-bit SISO used as a delay line delays data by **n clock periods**: 8 bits at 1 MHz → 8 µs. A register of n bits needs **n** flip-flops (one bit each), whatever its type.

Memory trick: **Shift left = × 2, shift right = ÷ 2** (unsigned): 0110 (6) ← 1100 (12). The name is "input-output": SIPO = **S**erial **I**n, **P**arallel **O**ut.

IT link: UART/SPI serial ports, CRC and pseudo-random generators (LFSR), the CPU's shift instructions (SHL/SHR), ring and Johnson counters.

12 Ripple (Asynchronous) & Synchronous Counters

Definition: A **counter** steps through a fixed sequence of states on clock pulses. **Modulus (mod)** = number of distinct states. n FFs → up to $\mathbf{2^n}$ states (count 0 … $2^n - 1$); a mod-N counter needs $\mathbf{\lceil \log_2 N \rceil}$ FFs and divides the clock frequency by N.



3-bit ripple up counter (falling-edge FFs): each stage toggles when the previous Q falls

FFs	Q → CLK	$\bar{Q}$ → CLK
−ve edge	**UP**	DOWN
+ve edge	DOWN	**UP**

Ripple delay: the last FF settles after $\mathbf{n \cdot t_{pd}}$ → $f_{max} = 1/(n \cdot t_{pd})$. 4-bit, t_{pd} = 10 ns → 40 ns → **25 MHz**. Ripple also causes decoding **glitches**.

Mod-N by reset: a NAND detects state N and drives the asynchronous CLR′. Mod-6: NAND(Q2, Q1) at 110. **Mod-10 (decade)**: NAND(Q3, Q1) at 1010. State N appears only for a few ns.

Synchronous counter: all FFs share CLK. T0 = 1, T1 = Q0, T2 = Q0Q1, T3 = Q0Q1Q2 (a bit toggles when all lower bits are 1). Down: use $\bar{Q}$s. $f_{max} = 1/(t_{pd} + t_{AND} + t_{su})$, **independent of n**.

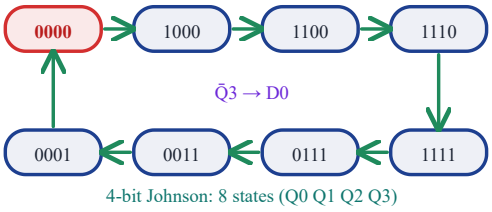
	Ripple (asynchronous)	Synchronous
Clocking	FF0 only; each Q clocks the next	one common CLK
Speed	slow ($n \cdot t_{pd}$), glitches	fast (one t_{pd}), no glitches
Hardware	simplest, no extra gates	extra AND gates
ICs	7490 (÷10), 7492 (÷12), 7493 (÷16)	74161, **74163**, 74193

Exam trap: 7490 = ÷2 + ÷5, counts BCD only with Q_A wired to CKB; 7493 = ÷2 + ÷8. 74161 clears **asynchronously**, 74163 **synchronously** (at the next edge); 74192/193 = up/down. A mod-N counter counts 0 to **N − 1**.

13 Ring & Johnson Counters

Ring counter: A shift register with **Q of the last FF → D of the first**, preset to 1000. One 1 circulates: 1000 → 0100 → 0010 → 0001. **n FFs → n states**.

Johnson counter: (twisted-ring, switch-tail): **$\bar{Q}$ of the last FF → D of the first**, start 0000. **n FFs → 2n states**.



4-bit Johnson: 8 states (Q0 Q1 Q2 Q3)

n FFs	Ring	Johnson	Binary
States	**n**	**2n**	2^n
Unused	$2^n - n$	$2^n - 2n$	0
Mod-10 FFs	10	5	4
Decoding	none	2-in AND	n-in AND
f_{out}	f/n	f/2n	$f/2^n$

Johnson decoding: one **2-input AND** per state, e.g. 0000 → Q0′Q3′, 1000 → Q0Q1′, 1111 → Q0Q3, 0001 → Q2′Q3. Neither counter is self-starting: a wrong power-up state circulates forever, so add **preset / correction logic**.

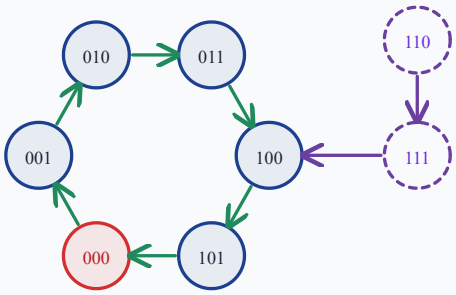
Memory trick: "**Ring = n, Johnson = 2n (the twist doubles).**" 4 FFs: ring mod-4, Johnson mod-8, binary mod-16.

14 Designing a Synchronous Counter

Worked example: mod-6 counter with D flip-flops ✍️:

1. FFs = $\lceil \log_2 6 \rceil$ = **3** (Q2 Q1 Q0)
2. State table 0 → … → 5 → 0; 110, 111 unused = X
3. Excitation: for D, **D = Q^+**
4. K-maps with X at m6, m7:
   D0 = **Q0′**
   D1 = Q1Q0′ + Q2′Q1′Q0
   D2 = Q2Q0′ + Q1Q0
5. Lock-out: 110 → 111 → 100 ✓ **self-starting**

Q2Q1Q0	next
000	001
001	010
010	011
011	100
100	101
101	000
11X	XXX



dashed = unused, both lead back in

With JK or T FFs the steps are the same; only step ③ uses the JK/T excitation table (a binary up counter gives $J_k = K_k = Q0 \cdot Q1 \cdots Q_{k-1}$).

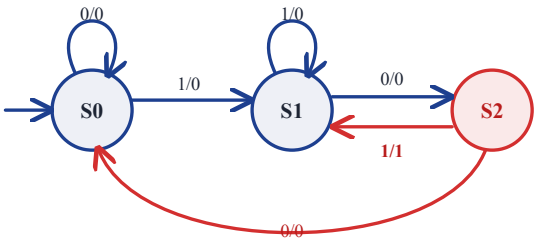
> **Exam trap:** **Lock-out**: a counter that powers up in an unused state and never reaches the main sequence. Always trace every unused state; if one loops on itself, change its X's to force a return (self-correcting design).

15 State Diagrams: Mealy vs Moore

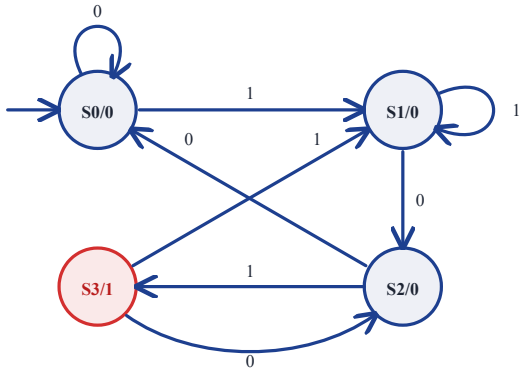
> **Definition:** A **finite state machine (FSM)** = states + inputs + outputs + next-state logic + output logic. A **state diagram** draws states as circles and transitions as arrows; a **state table** lists present state, input → next state, output. k states need **⌈log₂k⌉** FFs (binary code).

	Mealy	Moore
Output depends on	**present state + input**	**present state only**
Output written	on the arrow: **x/z**	in the circle: **S/z**
Number of states	fewer	usually more (often one more)
Response	same cycle; may glitch with input	a clock later; glitch-free

Sequence detector for 101 (overlapping): Mealy and Moore



Mealy: 3 states (S1 = "1", S2 = "10")



Moore: 4 states, output in the circle

Worked design (Mealy, D flip-flops) ✍️: Assign S0 = 00, S1 = 01, S2 = 10 (Q1Q0); 11 unused = X.

Q1Q0	x	$Q1^+Q0^+$	z
00	0 / 1	00 / 01	0 / 0
01	0 / 1	10 / 01	0 / 0
10	0 / 1	00 / 01	0 / **1**

K-maps (11 = X): **D1 = x′Q0 · D0 = x · z = xQ1**

Trace: x = 0 1 0 1 0 1 1 0 1 → z = 0 0 0 1 0 1 0 0 1

State reduction: two states are **equivalent** if, for every input, they give the same output and go to the same (or equivalent) next states; merge them (row matching,

State assignment: binary (fewest FFs), Gray (neighbouring states differ by 1 bit: simpler logic, race-free in async designs), **one-hot** (k FFs, one 1; fast,

implication chart). Fewer states → fewer FFs and gates.

used in FPGAs). The Moore detector above has z = Q1Q0 if S3 = 11.

Exam trap: **Overlapping** 101 detector: after 101 the last 1 starts the next match (S2 –1→ S1). Non-overlapping goes back to S0. "Output depends only on state" = **Moore**.

Night-Before-the-Exam Sheet

Fact sheet:

SR: $Q^+ = S + R'Q$ (SR = 0)

D: $Q^+ = D$ · T: $Q^+ = T \oplus Q$

JK excitation: 0→1 = 1X, 1→0 = X1

Race-around: J = K = 1, $t_p > t_{pd}$

Ripple $f_{max} = 1/(n \cdot t_{pd})$

Mod-N: $\lceil \log_2 N \rceil$ FFs, output f/N

Mealy: f(state, input) · Moore: f(state)

JK: $Q^+ = JQ' + K'Q$

NOR latch bad 11 · NAND latch bad 00

JK → T: J = K = T · D → T: D = T ⊕ Q

$f_{max} = 1/(t_{pd} + t_{comb} + t_{su})$

n FFs: binary 2^n · ring n · Johnson 2n

74194: 00 hold 01 SR 10 SL 11 load

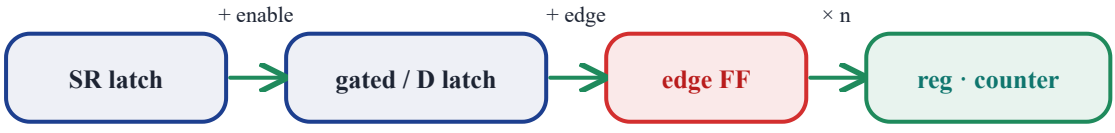
k states → $\lceil \log_2 k \rceil$ FFs (one-hot: k)

Top 12 one-liners for MCQs

1. Flip-flop = bistable multivibrator, 1 bit
2. Basic latch = 2 cross-coupled NOR/NAND
3. D FF has no invalid state; D latch is transparent
4. JK with J = K = 1 toggles
5. T FF with T = 1 divides f by 2
6. Master-slave cures race-around
7. Decade counter needs 4 FFs
8. 4-bit ripple, 16 kHz in → 1 kHz out
9. 7490 decade · 7493 4-bit · 74163 sync
10. 4-bit Johnson = mod-8, ring = mod-4
11. SISO: n clocks to load n bits
12. Moore needs ≥ states of Mealy

Top traps: NAND latch hold = 11, invalid = 00 · characteristic ≠ excitation · race-around only in **level**-triggered JK · ripple counter = asynchronous counter · mod-N counts to N − 1 · Johnson feeds back $\bar{Q}$, ring feeds back Q · setup is **before**, hold is **after** the edge

All tricks together: sequential = remembers · NOR hates two 1s, NAND hates two 0s · ▷ = edge, bubble ▷ = falling · D column = Q^+, T column = change · JK: Q = 0 → J decides, Q = 1 → K decides · D wants Q^+, T wants the change · ring n, Johnson 2n · shift left × 2



every block is built from the one before it

the build-up of the whole chapter

✓ Practice MCQs (exam pattern, 10 marks)

1. How many stable states does a flip-flop have?
(a) 0 (b) 1
(c) 2 (d) 3

2. Characteristic equation of the JK flip-flop:
(a) $JQ + KQ'$ (b) $JQ' + K'Q$
(c) $J + K'Q$ (d) $J \oplus Q$

3. A 4-bit Johnson counter has how many states?
(a) 4 (b) 16
(c) 6 (d) 8

4. Invalid input of a NAND ($\bar{S}\bar{R}$) latch:
(a) $\bar{S} = \bar{R} = 0$ (b) $\bar{S} = \bar{R} = 1$
(c) $\bar{S} = 0, \bar{R} = 1$ (d) $\bar{S} = 1, \bar{R} = 0$

5. Race-around (J = K = 1) occurs in a:
(a) D flip-flop (b) edge-triggered T
(c) level-triggered JK (d) SR latch

6. Minimum flip-flops for a mod-12 counter:
(a) 3 (b) 12
(c) 6 (d) 4

7. In the 74194, S1 S0 = 11 selects:
(a) parallel load (b) shift right
(c) shift left (d) hold

8. In a Moore machine the output depends on:
(a) inputs only (b) state + input
(c) state only (d) clock only

9. 3-bit ripple, t_{pd} = 20 ns/FF: $f_{max} \approx$
(a) 50 MHz (b) 16.7 MHz
(c) 25 MHz (d) 60 MHz

10. JK excitation for Q: 0 → 1 is:
(a) J = 1, K = X (b) J = X, K = 1
(c) J = 0, K = X (d) J = X, K = 0

Answer key → 1-c · 2-b · 3-d · 4-a · 5-c · 6-d · 7-a · 8-c · 9-b · 10-a

Written question: Design a 3-bit synchronous binary up counter using JK flip-flops. Give the state table, the excitation table and the J, K equations, and draw the circuit. 8 marks

1. 3 FFs ($2^3 = 8$ states); state table 000 → 001 → … → 111 → 000 — 1
2. For each bit write Q → Q^+ and fill J, K from the JK excitation table (0→0: 0X, 0→1: 1X, 1→0: X1, 1→1: X0) — 2
3. K-maps: **J0 = K0 = 1 · J1 = K1 = Q0 · J2 = K2 = Q0Q1** (bit k toggles when all lower bits are 1) — 3
4. Circuit: common CLK to all 3 FFs, FF0 tied to 1, one 2-input AND for Q0Q1; $f_{max} = 1/(t_{pd} + t_{AND} + t_{su})$ — 2

Before the exam ✓

- ✓ Can draw NOR and NAND latches with tables
- ✓ Know 4 characteristic + 4 excitation tables
- ✓ Can count states: binary, ring, Johnson
- ✓ Can design a counter and a 101 detector

Also practise (written):
① Convert JK → D and D → JK
② Mod-5 synchronous counter (JK)
③ 3-bit Johnson counter with decoding
④ Moore detector for 110